ControlPlane — Business Proposal

Accenture Innovation Challenge 2026 · Problem Track 1 · Team Nexus, IIT Jodhpur

Every number in this document that is not labelled an assumption comes from `eval/results/report.md`, produced by `eval/run_eval.py` over 319 labelled cases with a live judge model. The assumptions are listed in `docs/assumptions.md` rather than buried.

---

1. The problem

An enterprise runs generative AI across several use cases at once — a customer support assistant, an internal knowledge assistant, a decision-support tool inside a regulated workflow. Each one can be confidently wrong, quietly expensive, or subtly biased. The response reaches the user in two seconds. The review of that response reaches the enterprise in two weeks.

Three things make this harder than it first looks, and all three shaped what we built.

The risks overlap. A fabricated detail about a named customer is a hallucination and a privacy event at once. Systems that assign one label per response lose that. Ours does not: in testing, PII leaks co-fire with grounding failures on the same span, and the verdict records both.

There is no ground truth at runtime. The same knowledge gap that causes a hallucination makes it hard to check. What is available is the retrieved source the answer was supposed to be based on — so we verify the answer against its sources, and we are explicit that this does not catch a confidently wrong answer built on a wrong retrieval.

Over-flagging and under-flagging trade against each other. This cannot be solved, only priced. A missed hallucination in a regulated credit decision and a false alarm on an internal chatbot are not the same event, and any system that optimises a single accuracy score is pretending they are.

2. What we built

A verification layer that sits between the application and the model API, as a drop-in replacement for the model endpoint. Existing clients change a base URL.

The cascade

Each tier runs only when the previous one was not confident enough.

Tier	What it does	Cost	Measured latency
0	**Deterministic rules: PII patterns, access control, schema**	**free**	**under 1 ms**
1	**NLI entailment against each retrieved source, toxicity classifier, bias heuristic**	**GPU seconds**	**87–190 ms p95, by policy**
2	**LLM judge, only for claims inside the policy's uncertainty band**	**tokens**	**1.3–3.1 s**

Tier 2 fires on **2.8%** of responses. That number is not a target we set; it is what the band produces, and the eval sweeps the band to show what other settings would cost.

Tier 1 costs more where the policy asks for more: customer support runs per-chunk entailment at 87 ms p95, decision support adds the joined premise and pays 190 ms for it. That switch is in the YAML, and section 3 shows what it buys.

The action router

The router does not compare a score to a threshold. It weighs `P(wrong) × cost_of_being_wrong` against `cost_of_human_review` and picks the cheaper mistake. Every category is evaluated, so a response flagged for both privacy and grounding takes the more severe action and records both reasons.

The policy layer

No Python in this system knows what a use case is. Risk appetite, latency budget, uncertainty band, thresholds, actions, retention, jurisdiction and review capacity are all declared in YAML.

The same response, three policies, three verdicts:

```
hallucinated number   customer_support  -> regenerate   (being wrong costs Rs 400)
                      internal_copilot  -> annotate     (being wrong costs Rs 60)
                      decision_support  -> escalate     (being wrong costs Rs 50,000)
```

A new jurisdiction, a new use case, or a changed risk appetite is a config edit.

Verification beside the stream

Sentences are checked as they complete, while the model is still writing the next one. Tier 0 and tier 1 finish in well under the time a model takes to produce the following sentence, so on the clean path the verdict is ready when the last token is.

Tier 2 is not made to fit that shape. A judge call is 1.3–3.1 seconds against a 300 ms customer-support budget, so it runs after the stream closes and the policy decides whether the caller waits. When the text has already gone out, the gateway labels the late verdict **advisory** rather than applied — calling it enforcement would be untrue.

The audit trail

Every verdict and every human override is written to an append-only hash-chained log. Each record commits to the hash of the one before it, so a record cannot be altered afterwards without breaking the chain. Tampering is detected by test.

The feedback loop

Reviewed cases retune thresholds using the router's own arithmetic. Getting this to give a usable answer required fixing three things that each produced a confidently wrong one:

- **Censoring.** Reviewers only see what was flagged. Fit a threshold to the review queue and it says "flag everything" every time, because on that evidence passing is never observed to be safe. Policies now declare an `audit_sample_rate` and a slice of *passed* responses goes to review too.
- **Base rate.** Flagged cases are reviewed at 100% and sampled ones at 5%, so the queue looks far more dangerous than production. Records carry an inverse propensity weight.
- **Capacity.** With being wrong at Rs 50,000 against Rs 200 for a review, pure cost minimisation says review nearly everything. True, and useless. The sweep is constrained by the policy's declared review capacity.

With all three in place the loop found a real problem: on this set `decision_support` escalates around 32% of its traffic against its own declared 20% cap. It reports that and stops, because the fix is either more reviewers or more accepted risk, and neither is a threshold a tuning script should move on its own.

3. Results

319 labelled cases against a live judge (`openai/gpt-5.6-sol`). One call in 328 failed and fell back to the offline judge — 0.3%, counted and named in the report rather than left to be discovered.

Configuration	Catch rate	False positives	Tier 2 rate	p95 latency	Cost
Rules only	22.8%	0.0%	0%	0.02 ms	Rs 0
Rules + classifiers	93.5%	7.4%	0%	140 ms	Rs 0.35
The cascade	**94.6%**	**7.4%**	**2.8%**	**175 ms**	**Rs 1.10**
A judge on every response	92.4%	14.8%	100%	5534 ms	Rs 30.01

The cascade catches more than judging every response — 94.6% against 92.4% — at 3.7% of the cost, with half the false positives and 32× lower latency.

That the judge-everything baseline is *worse* on both is not a rhetorical win. A judge asked to re-rule on claims tier 1 already had right sometimes overrules them, and it does so in both directions. Selectivity is not only cheaper here, it is more accurate — measured against a live judge, not a stand-in.

What it cannot do

Stated because a benchmark that reports only its wins is a brochure.

Weakness	Rate	Why
Multi-hop claims	3 of 9	A claim true only by combining two sources entails neither alone
Quantifier flips	8 of 15 caught	"up to X" against "at least X" is one word and the opposite meaning
Hedged but correct	11 of 15	Hedging reads as distance from the source
Two policies over their own latency budget	`internal_copilot` 1225 ms vs 1000; `decision_support` 3914 ms vs 3000 when the judge runs	A judge call is seconds; a budget in hundreds of milliseconds cannot absorb one

The eval set is synthetic and was written by the same person who tuned the checker. An earlier version scored 100% on every case type, which is exactly why the adversarial cases exist. These numbers are a floor on difficulty, not a ceiling on quality.

4. Target users

Who	What they need	What they touch
Platform / ML engineering	**Ship AI features without owning risk review**	**A base URL change**
Risk, compliance, legal	**Evidence a decision was checked, and by what rule**	**The audit export and policy YAML**
Review operations	**A queue sized to their headcount**	**The escalation queue and capacity cap**
The business owner	**Cost per interaction that does not surprise them**	**The cost meter**

The buyer is usually the platform team; the renewal is signed by risk.

5. The business case

At the brief's reference volume of roughly 30,000 interactions a week — 1.56 million a year — using measured per-response costs:

	Per response	Per year
ControlPlane cascade	**Rs 0.0035**	**Rs 5,391**
A judge on every response	Rs 0.0941	Rs 146,763
Difference		**Rs 141,372**

The number that actually matters

Verification is not the expensive part. **Human review is.**

`decision_support` declares that it can review 20% of its traffic. Held to exactly that — the number the business itself budgeted for — one use case sends **104,000 responses a year** to a reviewer: **Rs 2.08 crore** in labour, against **Rs 5,391** of compute for the whole estate. Nearly four thousand times.

On our eval set it runs above that cap. Measured with inverse-propensity weighting over the reviewed queue, `decision_support` escalates **32.3%** — and 31.7% on a later run, the difference being the judge's non-determinism rather than noise in the measurement. At 32.3% the same use case sends **168,000 responses a year** to a reviewer: **Rs 3.36 crore** in labour, against Rs 5,391 of compute for the whole estate.

Both figures are real and they answer different questions. Rs 2.08 crore is what this policy budgeted for. Rs 3.36 crore is what it would actually spend if it kept escalating the way it does on our set — which is why the loop reports the overage instead of quietly moving the threshold.

Neither is a forecast, and we do not present one. The eval set is 57.7% harmful by construction, far more hostile than production traffic, so its escalation rate is an upper bound on what real traffic would produce. The argument does not depend on which number you take: review labour dominates verification by three to four orders of magnitude at any escalation rate above a fraction of a percent.

This reframes what the product is for. ControlPlane is not primarily a way to buy cheap verification; it is a way to **control how much human review you have to buy**, and to justify each unit of it. Moving the escalation rate by one percentage point is worth more than the entire verification bill.

That is why the router prices decisions instead of thresholding scores, and why the feedback loop optimises inside a declared review capacity.

Break-even

Verification costs Rs 0.0035 per response. At the measured catch rate it pays for itself when:

Use case	It pays for itself at
`internal_copilot`	1 harmful response in 16,400
`customer_support`	1 harmful response in 109,000
`decision_support`	1 harmful response in 13.7 million

Published hallucination rates for retrieval-grounded assistants are orders of magnitude above any of these. The verification layer is not a cost decision. The review queue is.

6. Roadmap

Phase 1 — done (this prototype). Three-tier cascade, policy layer, expected- cost router, cost meter with verified prices, hash-chained audit, OpenAI- compatible gateway, streaming-concurrent verification, feedback loop, 319-case benchmark with published failure modes.

Phase 2 — production readiness (1–2 quarters). Operator dashboard with a live escalation queue. Multi-tenant auth and RBAC. Retrieval-quality signals, so a grounded answer built on a stale chunk is distinguishable from a good one. Multi-hop grounding via evidence-set entailment. Calibration on customer data, replacing our assumed cost parameters with real ones.

Phase 3 — coverage (2–4 quarters). Multi-turn risk accumulation, where one questionable output shapes several downstream decisions. Agent action gating, verifying tool calls rather than only text. Counterfactual bias probing for disparate impact across a population, which shallow lexical detection cannot reach. Jurisdiction packs for EU AI Act and DPDP evidence export.

Phase 4 — scale. Distilled tier 1 models to cut the 85 ms. Regional inference for data residency. Continuous calibration from the review queue.

7. Risks

Risk	Why it is real	Mitigation
Cost assumptions are ours, not a customer's	**Every routing decision depends on `cost_of_being_wrong`, and we made it up**	**It is one YAML value per use case. Phase 2 calibrates it against real incident cost before any go-live**
Alert fatigue	**7.4% false positives at scale is a lot of noise, and people route around noisy tools**	**The tradeoff curve is published, escalation is capped by declared capacity, and the loop retunes from what reviewers actually decide**
Bad retrieval defeats grounding	**We check the answer against its sources, not the sources against the world**	**Named as a limit today. Retrieval-quality signals are Phase 2**
Bias detection is shallow	**Attribute-plus-decision heuristics miss bias expressed without either, and cannot see disparate impact**	**Scoped honestly, and the case for Phase 3 counterfactual probing**
The judge is a model too	**It can be wrong, and it costs money and seconds**	**It runs on 2.8% of traffic, its verdicts are logged with reasons, and the offline fallback keeps the system running when it fails**
Latency budgets are tight	**A judge call cannot fit inside 300 ms, and one policy already exceeds its budget**	**Tier 2 is out-of-band by design; late verdicts are labelled advisory rather than presented as enforcement**
Vendor lock-in	**Providers change prices and deprecate models**	**Two provider paths, prices in config with capture dates, and the gateway is provider-agnostic**
Our own benchmark flatters us	**We wrote the test set and tuned against it**	**Adversarial cases were added when it scored 100%; failure modes are published; Phase 2 validates on customer data**

8. Why this rather than the alternatives

Runtime guardrails exist. What does not exist is one layer where all three risk dimensions are scored together, each use case runs its own risk policy, and every check is priced.

Today a team stacks an eval framework, a grounding checker, an observability platform and an inline guardrail — four products, none of which can tell you whether the check it just ran was worth what it cost. ControlPlane's argument is that once verification is cheap, the real question is how much human attention to spend and where, and that is a question only a system that prices its own decisions can answer.

Prototype, benchmark and results: the public repository. Demo video: https://drive.google.com/file/d/1S4paAjOKEdX4FT1KDydjQjpdF2UseLI1/view?usp=sharing Every figure here is reproducible with `python3 eval/run_eval.py`.